9. From a given vertex in a weighted connected graph, find shortest path to other vertices using Dijkstra's algorithm

```
#include <stdio.h>
#include<limits.h>
#define MAX 100

int minDistance(int dist[], int visited[], int n){
    int min = INT_MAX, min_index = -1;

    for (int v=0; v<n; v++){
        if(visited[v]==0 && dist[v]<=min){
            min=dist[v];
            min_index = v;
        }
    }
    return min_index;
}

void dijkstra(int graph[MAX][MAX], int n, int src){
    int dist[MAX];
    int visited[MAX];

    for(int i=0; i<n; i++){
        dist[i] = INT_MAX;
        visited[i]=0;
    }
dist[src]=0;
```

```
for(int count=0; count<n-1; count++){
        int u = minDistance(dist, visited, n);
        visited[u] = 1;

        for(int v=0; v<n; v++){
            if(!visited[v] && graph[u][v] &&
                dist[u] != INT_MAX &&
                dist[u] + graph[u][v] < dist[v]){
                    dist[v] = dist[u] + graph[u][v];
            }
        }
    }

    printf("Vertex \t Distance from Source\n");
    for(int i=0; i<n; i++){
        printf("%d \t %d\n", i, dist[i]);
    }
}
int main(){
    int n, graph[MAX][MAX], src;
    printf("Enter number of vertices: ");
    scanf("%d", &n);
    printf("Enter adjacency matrix :\n");
    for(int i=0; i<n; i++)
        for(int j=0; j<n; j++)
            scanf("%d", &graph[i][j]);
    printf("Enter source vertex: ");
    scanf("%d", &src);
    dijkstra(graph, n, src);
    return 0;
}
```

Output:

```
Enter number of vertices : 4

Enter adjacency matrix :
1 0 0 1
0 0 1 1
1 1 1 1
0 0 0 0
Enter source vertex : 2
vertex          Distance from Source
0                       1
1                       1
2                       0
3                       1
```